Patrones de diseño.

Factory method:

Para seguir la “estructura” del factory method hay que hacer lo siguiente.

- Crear la interface inicial del objeto que queremos crear con los métodos que va a implementar.
- Hacer una clase creadora abstracta que contiene un atributo de la clase de la interface que hemos creado, un método protected abstracto que no retorna nada “factoryMethod()” y un método público que retorne el objeto de la interface
- Creamos nuestros productos concretos que queremos utilizar
- Después creamos los creadores concretos de los productos concretos que hemos creado. Estas clases se extenderán de la clase creadora abstracta; implementarán el factory method que retornará una instancia de ese producto concreto

Abstract factory

para cumplir con este patrón, se buscará crear familias de objetos abstractos implementados en una interface, con ello conseguimos que, por ejemplo, en el caso de que creemos una interface gráfica y no sepamos para qué tipo de sistema operativo va a ser, podamos agregar clases y “fábricas” concretas para cubrir que nuestro software cumpla con el nuevo sistema operativo, ya que este no tiene los mismos botones, formularios…

```
// 1. Interfaz de la Fábrica Abstracta
public interface GUIFactory {
    Boton crearBoton();
    Menu crearMenu();
}

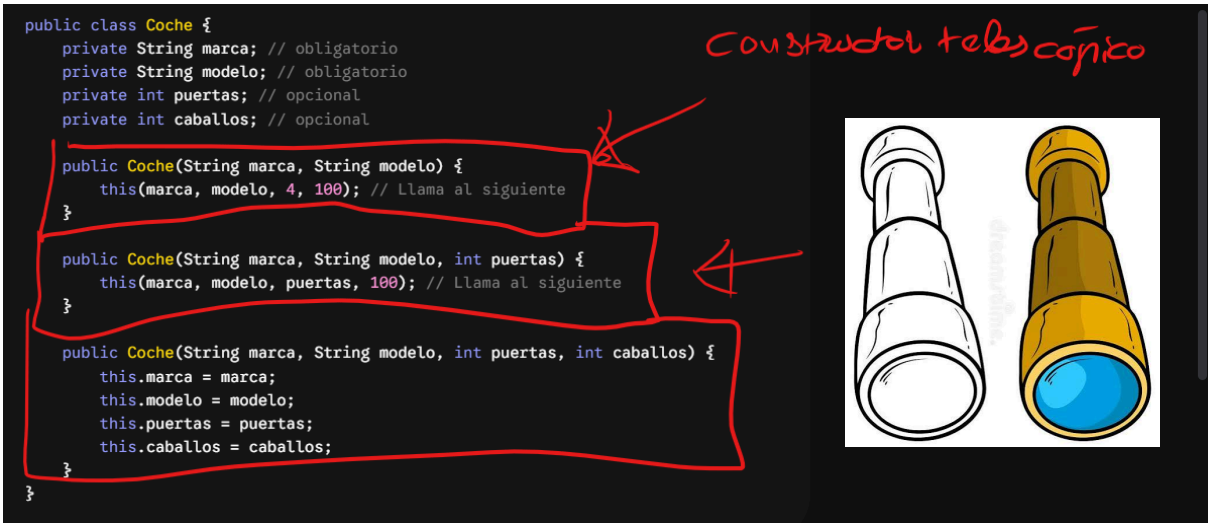
// 2. Fábrica Concreta para Windows
public class WindowsFactory implements GUIFactory {
    @Override
    public Boton crearBoton() {
        return new BotonWindows();
    }
    @Override
    public Menu crearMenu() {
        return new MenuWindows();
    }
}

// 3. Fábrica Concreta para macOS
public class MacFactory implements GUIFactory {
    @Override
```

```
    public Boton crearBoton() {
        return new BotonMac();
    }
    @Override
    public Menu crearMenu() {
        return new MenuMac();
    }
}
```

ANTIPATRÓN TELESCOPE

Para entender mejor por qué se usa builder, tenemos que conocer cuál es su antítesis es una mala práctica que es muy visual y reconocible.



Es fácil de detectar ya que es muy visual

PATRÓN BUILDER

Viene a solucionar una problemática clave, ya que no sería la primera vez que, si nuestra clase tiene muchos atributos, es muy tedioso crear un objeto teniendo que mirar qué parámetro va en qué lugar de la instanciación y usar el constructor. Para crearlo, necesitamos los siguientes requisitos:

- Creamos nuestra clase normal
- Dentro de ella creamos una clase anidada (una clase dentro de otra), la llamaremos builder. Tiene que ser estática para poder acceder a ella desde la clase normal que hemos creado.
- Le ponemos todos los atributos de la clase normal. Importante: Los atributos deberían ser privados para que solo pueda acceder la clase builder, pero no deberían ser final, ya que la propia clase va a modificarlos.
- Le ponemos los setters de todos sus atributos y, muy importante, retornarán al propio objeto builder usando this.

- Ahora creamos el método build(), este método retornará un objeto de la clase normal creando una instancia de él pasándole el objeto builder con this. Esto lo hacemos principalmente porque crearemos un constructor en la clase principal que acepte un objeto builder.
- Crear un constructor de la clase principal que acepte un objeto builder e instanciamos sus variables igualando el resultado de los valores de nuestro objeto builder, ya que son los mismos que la clase principal.

Detalles importantes de este patrón:

- Es bastante común o el “estándar” quitarle la palabra set a los setters de la clase builder porque así es más legible.
- También tiene una utilidad bastante útil, que es la de añadir restricciones a la hora de crear objetos en el método build; por ejemplo, con una condicional, podemos decirle que si el atributo String nombre es nulo, lo instancie como “”.

Esto hace más legible la instanciación de un objeto; siempre tenemos que llamar al método build cuando terminemos.

Aclaraciones posteriores sobre builder: builder también soluciona la problemática a la hora de instanciar un objeto que tiene variables que necesitan de una instanciación obligatoria y las que tienen valores por defecto.

Singleton

requisitos:

- Un atributo o propiedad privado estático del tipo de la clase singleton que se llame instancia (así será más legible e interpretable)
- Un constructor privado
- Un método getInstancia que retorne el tipo de la clase singleton y que verifique si la variable instancia está vacía (en ese caso creará la instancia) o si ya está creada, devolverá la que ya existe.

Patrón Adapter

Vamos a partir de una clase que trabaja con un tipo de dato y que ese dato a día de hoy ha cambiado. Respetando el principio SOLID Open & Closed, no queremos tocar esa clase porque nos sigue valiendo la utilidad que hace en el programa. Por ejemplo, imaginemos que en una clase que se encarga de hacer el cobro bancario por comprar un producto “pasarela de pago”, en un principio es una pasarela que está anticuada en su método para efectuar el pago; recibe un XML que contiene los datos del usuario: nombre, contraseña…

```
public class PasarelaAntiguaBancaria {
    // Exige un formato XML en formato String:
"<pago><cliente>ID</cliente><monto>VALOR</monto></pago>"
    public void hacerCobroXml(String xmlData) {
        System.out.println("Procesando pago mediante pasarela antigua con XML:\n" +
xmlData);
    }
```

}

Esto es lo que soluciona el patrón Adapter: problemas de compatibilidad entre clases.

¿Cuáles son los requisitos de este patrón? Según veo, tiene que ser una propiedad con la clase antigua que acepta el tipo de dato viejo, que la clase implemente el método de una interfaz que recibe el nuevo tipo de dato y en ese método hacer la transformación de datos y el constructor, claro, que no se nos olvide.

La definición más clara que puede ayudar a entender el patrón adapter es que funciona o es como un traductor.

PATRÓN FACADE

Según lo entiendo yo, es una manera de, a través de la herencia de un objeto, crear objetos específicos de él en los constructores de las clases concretas; luego los metemos todos en la fachada para acceder a cada uno, haciendo que todas las funcionalidades de un campo específico del programa estén juntas y podamos acceder a ellas de manera más sencilla.

Ej. de IA que me pareció interesante:

Imagina la fachada de un edificio o la recepción de un hotel:

- En un hotel, si quieres reservar una habitación, pedir comida a la habitación, mandar a lavar ropa y contratar un taxi, **no vas a hablar directamente** con el limpiador, el cocinero, el chofer y el encargado de la lavandería.
- Vas a la **recepción (la Fachada)** y le pides todo a la persona del mostrador. Él o ella se encarga de coordinar con todos los subsistemas internos sin que tú tengas que preocuparte por cómo se comunican entre sí.